Tomasulo Algorithm Implementation

Ali Alfageeh
Computer Architecture - Programming Assignment
October 25, 2026

GitHub repository: https://github.com/fqeh/CompArchAssignment1

1 Introduction

In this assignment, I extended a Python simulator of Tomasulo's algorithm to issue one, two, three, or four instructions per cycle. My goal was to compare how issue width affects performance while keeping the simulated program's results correct. I kept one common data bus and one in-order commit per cycle so that the comparison focuses on the effect of wider issue.

I used reservation stations and register renaming to track dependencies and allow ready instructions to execute out of order. I used a reorder buffer to commit completed instructions in program order. I also made the processor configuration and initial register and memory values readable from an input file, so I could run different programs without editing the simulator.

Main changes I made

I changed the issue stage to handle several instructions in the same cycle and corrected how dependencies and register mappings are tracked. I revised execution timing and resource handling so that instructions wait for their operands and use the functional units consistently.

I added branch prediction and recovery so the simulator can continue issuing instructions before a branch is resolved and discard incorrect speculative work when needed. I also improved load and store handling, including forwarding, memory-order recovery, unaligned addresses, and single-precision memory transfers.

I expanded the output to show each instruction's pipeline timing, the final register and memory values, and execution statistics. In the following sections, I explain the main code changes and compare measured results at each issue width.

How I checked my implementation

I checked the final architectural state against an independent sequential interpreter and reviewed instruction timing and recovery behavior. My validation included 38 passing test methods, 64 benchmark runs, and 160 randomized comparisons. I also ran the two sample workloads at all four issue widths and included their timing tables in this report.

2.1 Configuration and issue-width input

The starter reads only an instruction file and hardcodes processor state. The revised parser accepts the hardware table, ROB entries, register/memory initialization, and Issue width. The default is 1; values outside 1-4 are rejected. The command-line option can override the input.

Initial version

BEFORE | original/tomasulo-python/code/main.py | lines 15-26

```
15  instructions = read_instruction('code.in')
16  '''initialize regs, RAT, memory'''
17  # integer REG
18  reg_int = []
19  for i in range(32):
20      reg_int.extend([0])
21  reg_int[1] = 12
22  reg_int[2] = 32
23  rat_int = deepcopy(reg_int)
24  # fp REG
25  reg_fp = []
26  for i in range(32):
```

My version

AFTER | code/init.py | lines 101-114

```
101         match = re.fullmatch(r'(rob\s+entries|issue\s+width)\s*=\s*(-?\d+)', low)
102         if match:
103             value = int(match[2])
104             if match[1].startswith('rob'):
105                 if value < 1:
106                     raise ValueError('ROB entries must be positive')
107                 cfg.rob_entries = value
108             else:
109                 if not 1 <= value <= 4:
110                     raise ValueError('Issue width must be between 1 and 4')
111                 cfg.issue_width = value
112             continue
113         if '=' in line:
114             for assignment in line.split(','):
```

Figure 1. Before-and-after source snapshots. The snapshots show the original fixed setup and the new validated ROB/issue-width configuration block. Hardware-row parsing appears immediately above the modified excerpt.

2.2 In-order multiple issue

The issue stage now repeats up to the configured width. It checks the ROB and the appropriate reservation station or load/store queue before each instruction. A structural hazard stops the bundle. Capturing each source before updating the destination mapping preserves dependencies between instructions issued in the same cycle.

Initial version

BEFORE | original/tomasulo-python/code/issue.py | lines 171-181

```
171 def issue(cycle, PC, instructions, ROB, size_ROB,
172         rs_int_adder,
173         rs_fp_adder,
174         rs_fp_multi,
175         ld_sd_queue, size_ld_sd_queue,
176         rat_int, rat_fp):
177     # fetch 1 instruction
178     ins = instructions[PC.PC]
179     op = ins.split(' ')[0]
180     '''decode'''
181     # LD/SD instructions
```

My version

AFTER | code/issue.py | lines 5-26

```
5  def issue(sim):
6      if sim.cycle < sim.fetch_after:
7          return
8      for _ in range(sim.width):
9          if not 0 <= sim.pc < len(sim.program.instructions):
10             break
11         ins = sim.program.instructions[sim.pc]
12         queue = sim.lsq if ins.unit == 'load_store' else sim.rs[ins.unit]
13         capacity = getattr(sim.config, ins.unit).rs
14         if len(sim.rob) >= sim.config.rob_entries or len(queue) >= capacity:
15             sim.stats['structural_stalls'] += 1
16             break  # Never bypass a blocked instruction in an issue bundle.
17         # Capture sources before renaming destination, including within bundle.
18         operands = [sim.source(register) for register in ins.sources]
19         entry = Entry(sim.next_id, sim.pc, ins, sim.cycle, operands)
20         sim.next_id += 1
21         sim.rob.append(entry)
22         sim.history.append(entry)
23         queue.append(entry)
24         sim.stats['issued'] += 1
25         if ins.dest and ins.dest != 'R0':
26             sim.rat[ins.dest] = entry.id
```

Figure 2. Before-and-after source snapshots. A blocked instruction is never skipped. Each successful issue updates the RAT before the next issue slot is considered. The branch-aware PC selection follows this excerpt.

2.3 Safe renaming and single-CDB write-back

The original broadcast overwrites the register alias entry with a value even when a younger instruction has already renamed that register. The revision leaves the latest producer tag in the RAT and broadcasts only to operands waiting for the matching dynamic ID. Write-back chooses one eligible completed result per cycle.

Initial version

BEFORE | original/tomasulo-python/code/wb.py | lines 26-33

```
26      # rat_int
27      if reg_tag[0] == 'R':
28          rat_int[int(reg_tag[1:])] = cdb.value
29      # rat_fp
30      if reg_tag[0] == 'F':
31          rat_fp[int(reg_tag[1:])] = cdb.value
32      # rs list
33      rs_list = [rs_int_adder, rs_fp_adder, rs_fp_multi]
```

My version

AFTER | code/wb.py | lines 4-19

```
4   def wb(sim):
5       eligible = [r for r in sim.results if r[0] < sim.cycle]
6       if not eligible:
7           return
8       result = min(eligible, key=lambda r: (r[0], r[1]))
9       sim.results.remove(result)
10      entry = result[2]
11      entry.wb = sim.cycle
12      # Tags remain in the RAT until their producer commits. An older WAW
13      # producer therefore cannot overwrite a newer register mapping.
14      for waiting in sim.rob:
15          for operand in waiting.operands:
16              if operand.tag == entry.id:
17                  operand.tag = None
18                  operand.value = entry.value
19                  operand.ready_at = sim.cycle
```

Figure 3. Before-and-after source snapshots. An older WAW producer cannot erase a younger mapping. At commit, a mapping is removed only if its tag still equals the retiring entry ID. Sources from already-broadcast producers can be read from the ROB.

2.4 Execution timing and pipeline resources

The starter advances functional-unit counters inside a routine that can be called twice in one cycle. I replaced incremental advancement with explicit inclusive start/end cycles. Ready instructions may bypass older blocked instructions in execution, but they cannot start in their issue cycle or consume a value broadcast during the same cycle.

Initial version

BEFORE | original/tomasulo-python/code/exe.py | lines 50-60

```
50  def fu_exe(fu, fu_results, ROB, time_fu, cycle, PC):
51      if len(fu)!=0:
52          for element in fu:
53              element.cycle += 1
54          # write starting cycle
55          if fu[-1].cycle == 1:
56              index = find_ROB_entry(ROB, fu[-1].dest_tag)
57              ROB[index].exe.append(cycle)
58          # finish cycle
59          if fu[0].cycle == time_fu:
60              index = find_ROB_entry(ROB, fu[0].dest_tag)
```

My version

AFTER | code/exe.py | lines 24-37

```
24              if not sim.active(entry) or entry.ex or entry.issue >= sim.cycle:
25                  continue
26              operands = entry.operands[:1] if unit == 'load_store' else entry.operands
27              if not all(sim.ready(operand) for operand in operands):
28                  continue
29              entry.ex = [sim.cycle, sim.cycle + cfg.ex - 1]
30              job = dict(entry=entry, unit=unit, end=entry.ex[-1])
31              sim.jobs.append(job)
32              if unit != 'load_store':
33                  queue.remove(entry)
34              slots -= 1
35              if job['end'] == sim.cycle:
36                  complete(sim, entry)
37      sim.jobs = [j for j in sim.jobs if j['end'] > sim.cycle and sim.active(j['entry'])]
```

Figure 4. Before-and-after source snapshots. Each integer/address unit remains occupied through its completion cycle. Each FP unit can launch one operation per cycle. The shown end-cycle calculation prevents accidental double advancement.

2.5 Branch prediction at issue

The starter stalls fetching on Bne and has no prediction. The new issue logic looks up an eight-entry tagged branch-target buffer using the least significant three bits of the word PC. An absent/mismatched entry predicts not taken; a taken hit redirects the next issue slot. Beq and Bne are both implemented.

Initial version

BEFORE | original/tomasulo-python/code/issue.py | lines 234-247

```
    # Bne
    elif (op=='Bne'):
        # check space
        if (len(ROB)<size_ROB)&(check_rs_space(rs_int_adder)>=0):
            # put ins into ROB
            put_ins_into_ROB(ROB, size_ROB, PC, cycle, ins, '')
            # put ins into rs_int_adder
            index = check_rs_space(rs_int_adder)
            put_bne_into_rs(rs_int_adder, index, ins, rat_int, rat_fp, ROB)
            # PC
            PC.valid = 0
    else:
        pass

```

My version

AFTER | code/issue.py | lines 27-34

```
        next_pc = sim.pc + 1
        if ins.op in ('beq', 'bne'):
            prediction = sim.btb[sim.pc & 7]
            if prediction is not None and prediction['pc'] == sim.pc and prediction['taken']:
                next_pc = prediction['target']
            entry.predicted_next = next_pc
            sim.stats['branch_predictions'] += 1
        sim.pc = next_pc
```

Figure 5. Before-and-after source snapshots. Prediction is recorded per dynamic branch. Resolution at the end of EX compares the actual next PC with this prediction and updates the BTB direction bit and target.

2.6 Misprediction recovery

The starter changes the PC when a branch resolves, without speculative-state recovery. The new recovery path marks younger entries squashed, removes their reservation stations, queued results, executing jobs and memory requests, and rebuilds the RAT from the surviving ROB. Correct-path issue resumes at cycle n+2 after a misprediction resolved in n.

Initial version

BEFORE | original/tomasulo-python/code/exe.py | lines 71-82

```
            elif (fu[0].op=='Bne'):
                fu_results.pop()
                if fu[0].value1 == fu[0].value2:
                    PC.PC += 1
                    PC.valid = 1
                else:
                    index = find_ROB_entry(ROB, fu[0].dest_tag)
                    offset = ROB[index].dest_tag
                    PC.PC = int(PC.PC + 1 + offset/4)
                    PC.valid = 1
            else:
                pass
```

My version

AFTER | code/simulator.py | lines 90-110

```
    def recover(self, first_id, correct_pc, reason):
        """Remove every younger effect and rebuild renaming from surviving ROB."""
        for entry in self.rob:
            if entry.id >= first_id:
                entry.status = 'squashed'
                entry.squash_reason = reason
        self.rob = [e for e in self.rob if self.active(e)]
        self.lsq = [e for e in self.lsq if self.active(e)]
        for unit in self.rs:
            self.rs[unit] = [e for e in self.rs[unit] if self.active(e)]
        self.jobs = [job for job in self.jobs if self.active(job['entry'])]
        self.results = [result for result in self.results if self.active(result[2])]
        if self.port and not self.active(self.port['entry']):
            self.port = None
        self.rat.clear()
        for entry in self.rob:
            if entry.ins.dest and entry.ins.dest != 'R0':
                self.rat[entry.ins.dest] = entry.id
        self.pc = correct_pc
        self.fetch_after = self.cycle + 2

```

Figure 6. Before-and-after source snapshots. Rebuilding mappings in surviving program order preserves the youngest valid writer to each register. Dynamic IDs are never reused, preventing old broadcasts from waking new instructions.

2.7 Load/store forwarding and speculative loads

The original forwarding loop stops at the nearest store even when its address does not match. The revised memory stage selects the youngest older source for each of the four load bytes. It waits only for contributing store data. Full byte coverage forwards in one cycle; partial coverage is merged with memory at the configured latency. Loads can speculate past unresolved older store addresses.

Initial version

BEFORE | original/tomasulo-python/code/mem.py | lines 19-30

```
19  def forward_check_from_sd(ld_sd_queue, index):
20      data = []
21      for i in reversed(range(index)):
22          if (ld_sd_queue[i].op=='Sd'):
23              if (ld_sd_queue[i].address == ld_sd_queue[index].address):
24                  if (type(ld_sd_queue[i].data)==int)|(type(ld_sd_queue[i].data)==float):
25                      data.append(ld_sd_queue[i].data)
26              break
27      return data
28
29  # function: check if previous all Sds are ready and un-matched
30  def check_all_previous_Sd_no_match(ld_sd_queue, index):
```

My version

AFTER | code/mem.py | lines 45-57

```
45  def select_byte_sources(sim, load):
46      """For each load byte, select its youngest known older overlapping store."""
47      sources = [None] * 4
48      for store in reversed(sim.lsq):
49          if store.id >= load.id or store.ins.op != 'sd' or store.address is None:
50              continue
51          first = max(store.address, load.address)
52          last = min(store.address + 4, load.address + 4)
53          for address in range(first, last):
54              offset = address - load.address
55              if sources[offset] is None:
56                  sources[offset] = store
57      return sources
```

Figure 7. Before-and-after source snapshots. Forwarding and memory accesses share one serialized path. Per-byte source IDs remain in the ROB after load completion, allowing a later-discovered overlap to trigger replay without falsely replaying bytes hidden by newer stores.

2.8 Precise store commit and architectural updates

The starter tests the truth value of a function object instead of calling the store-committable check. The replacement permits a store memory operation only when that store is the ROB head. It holds the port for the configured memory latency, writes memory on the final cycle, and then retires. An uncommitted wrong-path store cannot change architectural memory.

Initial version

BEFORE | original/tomasulo-python/code/mem.py | lines 127-136

```
        if (ld_sd_queue[0].op=='Sd'):
            # check if this Sd is ready and to be commited
            if (ld_sd_queue[0].ready==1):
                # check to be committed in next cycle
                if (check_if_sd_committable):
                    # already have data in Sd
                    if (type(ld_sd_queue[0].data)==int)|(type(ld_sd_queue[0].data)==float):
                        # put the Sd entry into ld_sd_mem
                        entry = ld_sd_queue.popleft()
                        put_entry_into_ld_sd_mem(ld_sd_mem, entry)
```

My version

AFTER | code/commit.py | lines 4-26

```
def commit(sim):
    if not sim.rob:
        return
    entry = sim.rob[0]
    if entry.fault:
        raise ValueError(f'Instruction {entry.id} ({entry.ins.text}): {entry.fault}')
    if entry.ins.op == 'sd':
        if not entry.ex or entry.ex[-1] >= sim.cycle or not sim.ready(entry.operands[1]):
            return
        if not entry.mem:
            if sim.port is not None and sim.port['end'] >= sim.cycle:
                return
            end = sim.cycle + sim.config.memory_cycles - 1
            entry.mem = [sim.cycle, end]
            sim.port = dict(entry=entry, end=end, kind='store')
        if entry.mem[-1] != sim.cycle:
            return
        # Only the ROB head can update memory, at its retirement cycle.
        sim.memory[entry.address] = entry.operands[1].value
        sim.lsq.remove(entry)
    elif entry.ins.op in ('beq', 'bne'):
        if entry.resolved_at is None or entry.resolved_at >= sim.cycle:
            return
```

Figure 8. Before-and-after source snapshots. Store MEM ranges represent commit-time memory access. Branches and stores have no CDB WB entry. One instruction commits per cycle; a register-writing instruction can commit only after its WB cycle.

2.9 Accepting byte address 18

The previous version incorrectly required four-byte alignment. The assignment never states that restriction and its own example computes address 18 from R1=10 and Ld F4,8(R1). I removed the alignment requirement and replaced independent word entries with one shared 256-byte memory. Any four-byte start from 0 through 252 is valid; overlapping accesses observe the same bytes.

Initial version

BEFORE | revision_before/init.py | lines 172-174

```
172 def validate_address(address):
173     if not isinstance(address, int) or address % 4 or not 0 <= address <= 252:
174         raise ValueError(f'Address {address} must be word-aligned in 0..252')
```

My version

AFTER | code/init.py | lines 174-176

```
174 def validate_address(address):
175     if not isinstance(address, int) or not 0 <= address <= 252:
176         raise ValueError(f'Address {address} must be an integer in 0..252')
```

Figure 9. Before-and-after source snapshots. This comparison uses my earlier alignment check. Range checking remains: a four-byte access beginning at 253 would extend beyond byte 255.

Shared-byte storage used by the revised version

AFTER | code/init.py | lines 205-216

```
205     def __getitem__(self, address):
206         return unpack_float32(self.read_bytes(address))
207 
208     def __setitem__(self, address, value):
209         validate_address(address)
210         self.data[address:address + 4] = pack_float32(value)
211         self.view_starts.add(address)
212 
213     def read_bytes(self, address):
214         validate_address(address)
215         return bytes(self.data[address:address + 4])
216 
```

The assignment does not specify endianness or extra unaligned latency. The model uses little-endian storage and the configured memory latency for every valid start address. Memory views are decoded from these shared bytes.

2.10 Single-precision memory transfers

The previous version copied Python values directly into memory. The revised memory setter encodes each stored value as four binary32 bytes, and loads decode those bytes. Store-to-load forwarding uses the same encoding before merging bytes, so bypassing memory cannot bypass rounding. FP register initialization and .d arithmetic keep their existing precision because the assignment does not fully define their rounding rules.

Initial version

BEFORE | revision_before/commit.py | lines 21-23

```
21          # Only the ROB head can update memory, at its retirement cycle.
22          sim.memory[entry.address] = entry.operands[1].value
23          sim.lsq.remove(entry)
```

My version

AFTER | code/init.py | lines 179-193

```
179 def pack_float32(value):
180     """Round a value to four little-endian IEEE 754 binary32 bytes.
181 
182     Python struct applies round-to-nearest, ties-to-even. Values too large for
183     finite binary32 become signed infinity, rather than aborting a store.
184     """
185     try:
186         return struct.pack('<f', value)
187     except OverflowError:
188         return struct.pack('<f', math.copysign(math.inf, value))
189 
190 
191 def unpack_float32(data):
192     return struct.unpack('<f', data)[0]
193 
```

Figure 10. Before-and-after source snapshots. The shared memory setter in Figure 9 calls pack_float32; mem.py uses the same helper for forwarding. Stores round to nearest, ties to even; finite overflow becomes signed infinity. This replaces the previous limitation for loads/stores, without imposing binary32 arithmetic everywhere.

Numerical regression

```
F1=16777217
Sd F1, 18(R0)
Ld F2, 18(R0)
```

Expected and measured: F1 retains 16777217; F2 becomes 16777216 after the single-precision transfer. A forwarded load and a later memory load must return the same rounded value. Signed zero, subnormals, partial overlap and boundary accesses are covered by the updated tests.

3 Results

All results below were regenerated after the memory corrections. For each benchmark, widths 1, 2, 3 and 4 use identical instructions, hardware configuration and initial state. Only issue width changes. The independent sequential interpreter verifies final registers, decoded memory views, all 256 raw memory bytes, committed instruction count, and the committed PC sequence.

3.1 Cycle-count comparison

Benchmark	Width 1	Width 2	Width 3	Width 4	Check
address18	14	13	13	13	PASS
binary32_transfer	18	18	18	18	PASS
branch_squash	13	13	13	13	PASS
btb_collision_loop	43	43	43	43	PASS
byte_overlap	22	22	22	22	PASS
case_and_boundaries	19	19	19	19	PASS
daxpy_loop	66	65	65	65	PASS
memory_replay	25	25	25	25	PASS
parallel_fp	19	19	19	19	PASS
partial_overlap_replay	25	25	25	25	PASS
rename_hazards	25	22	21	20	PASS
sample_01	36	36	36	36	PASS
sample_02	47	46	46	46	PASS
starter_equivalent	45	45	45	45	PASS
store_forwarding	21	21	21	21	PASS
structural_stalls	24	24	24	24	PASS

Interpretation

The rename-hazard workload improves from 25 to 20 cycles, a **1.25x speedup** (20% fewer cycles). Sample 02 improves from 47 to 46 cycles, approximately **1.022x**. Sample 01 remains at 36 cycles. Wider issue can expose ready work sooner, but it does not remove the single-CDB, in-order-commit, memory-port, or dependency bottlenecks.

IPC = committed instructions / total cycles. Speedup = width-1 cycles / selected-width cycles. Squashed instructions consume resources and appear in the simulator output, but do not count as committed instructions. I used the same input program and initial state for all four widths in each comparison.

Timing conventions

Stage processing order is COMMIT, WB, MEM, EX, ISSUE. EX and MEM intervals are inclusive; an interval such as 10-14 is five cycles. A dash denotes a stage not used. Readiness timestamps require consumers to start after the producing WB cycle. The following sample tables contain every committed instruction and use the same columns as the supplied report.

3.2 Test case #01

This workload and its configuration are transcribed from the supplied sample report. The timing results on the following pages were recomputed with this implementation.

Unit	RS	EX cycles	MEM cycles	FUs
Integer adder	4	1	-	1
FP adder	4	3	-	1
FP multiplier	4	5	-	1
Load/store unit	4	1	6	1

ROB entries = 64; issue width varied from 1 through 4.

```
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0
```

Instruction list

```
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
```

Final state at every width: R1=28, R2=32, F1=5, F2=25, F3=0, F4=26, F20=3. R0 remains zero; all other registers are zero. Nonzero memory remains unchanged from the six initialized locations shown above.

Manual check: the three loads access addresses 12, 20 and 28, whose values are 1, 0 and 5. Thus F4 = 1 squared + 0 squared + 5 squared = 26. F0 is zero, so the other product accumulation leaves F3 = 0.

Test case #01: widths 1 and 2

Issue width 1 - 36 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-8	9	10
Mult.d F2 F1 F0	2	10-14	-	15	16
Add.d F3 F3 F2	3	16-18	-	19	20
Mult.d F2 F1 F1	4	11-15	-	16	21
Add.d F4 F2 F4	5	17-19	-	20	22
Addi R1 R1 8	6	7	-	8	23
Ld F1 0(R1)	7	9	10-15	17	24
Mult.d F2 F1 F0	8	18-22	-	23	25
Add.d F3 F3 F2	9	24-26	-	27	28
Mult.d F2 F1 F1	10	19-23	-	24	29
Add.d F4 F2 F4	11	25-27	-	28	30
Addi R1 R1 8	12	13	-	14	31
Ld F1 0(R1)	13	15	16-21	22	32
Mult.d F2 F1 F0	14	23-27	-	29	33
Add.d F3 F3 F2	16	30-32	-	33	34
Mult.d F2 F1 F1	17	24-28	-	30	35
Add.d F4 F2 F4	18	31-33	-	34	36

Issue width 2 - 36 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-8	9	10
Mult.d F2 F1 F0	1	10-14	-	15	16
Add.d F3 F3 F2	2	16-18	-	19	20
Mult.d F2 F1 F1	2	11-15	-	17	21
Add.d F4 F2 F4	3	18-20	-	21	22
Addi R1 R1 8	3	4	-	5	23
Ld F1 0(R1)	4	6	9-14	16	24
Mult.d F2 F1 F0	4	17-21	-	23	25
Add.d F3 F3 F2	5	24-26	-	27	28
Mult.d F2 F1 F1	5	18-22	-	24	29
Add.d F4 F2 F4	6	25-27	-	28	30
Addi R1 R1 8	6	7	-	8	31
Ld F1 0(R1)	7	9	15-20	22	32
Mult.d F2 F1 F0	10	23-27	-	29	33
Add.d F3 F3 F2	16	30-32	-	33	34
Mult.d F2 F1 F1	16	24-28	-	30	35
Add.d F4 F2 F4	18	31-33	-	34	36

I measured these timings using sample_01.in at issue widths 1 and 2. EX/MEM are inclusive cycle ranges. All 17 instructions commit in program order.

Test case #01: widths 3 and 4

Issue width 3 - 36 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-8	9	10
Mult.d F2 F1 F0	1	10-14	-	15	16
Add.d F3 F3 F2	1	16-18	-	19	20
Mult.d F2 F1 F1	2	11-15	-	17	21
Add.d F4 F2 F4	2	18-20	-	21	22
Addi R1 R1 8	2	3	-	4	23
Ld F1 0(R1)	3	5	9-14	16	24
Mult.d F2 F1 F0	3	17-21	-	23	25
Add.d F3 F3 F2	3	24-26	-	27	28
Mult.d F2 F1 F1	4	18-22	-	24	29
Add.d F4 F2 F4	4	25-27	-	28	30
Addi R1 R1 8	4	5	-	6	31
Ld F1 0(R1)	5	7	15-20	22	32
Mult.d F2 F1 F0	10	23-27	-	29	33
Add.d F3 F3 F2	16	30-32	-	33	34
Mult.d F2 F1 F1	16	24-28	-	30	35
Add.d F4 F2 F4	18	31-33	-	34	36

Issue width 4 - 36 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-8	9	10
Mult.d F2 F1 F0	1	10-14	-	15	16
Add.d F3 F3 F2	1	16-18	-	19	20
Mult.d F2 F1 F1	1	11-15	-	17	21
Add.d F4 F2 F4	2	18-20	-	21	22
Addi R1 R1 8	2	3	-	4	23
Ld F1 0(R1)	2	5	9-14	16	24
Mult.d F2 F1 F0	2	17-21	-	23	25
Add.d F3 F3 F2	3	24-26	-	27	28
Mult.d F2 F1 F1	3	18-22	-	24	29
Add.d F4 F2 F4	3	25-27	-	28	30
Addi R1 R1 8	3	5	-	6	31
Ld F1 0(R1)	4	7	15-20	22	32
Mult.d F2 F1 F0	10	23-27	-	29	33
Add.d F3 F3 F2	16	30-32	-	33	34
Mult.d F2 F1 F1	16	24-28	-	30	35
Add.d F4 F2 F4	18	31-33	-	34	36

I measured these timings using sample_01.in at issue widths 3 and 4. EX/MEM are inclusive cycle ranges. All 17 instructions commit in program order.

3.3 Test case #02

This workload and its configuration are transcribed from the supplied sample report. The timing results on the following pages were recomputed with this implementation.

Unit	RS	EX cycles	MEM cycles	FUs
Integer adder	4	1	-	1
FP adder	3	3	-	2
FP multiplier	2	15	-	2
Load/store unit	4	1	5	2

ROB entries = 128; issue width varied from 1 through 4.

```
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0
```

Instruction list

```
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Ld F1 0(R1)
Addi R1 R1 4
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
```

Final state at every width: R1=24, R2=32, F1=6, F2=36, F3=0, F4=37, F20=3. R0 remains zero; all other registers are zero. Nonzero memory remains unchanged from the six initialized locations shown above.

Manual check: the accumulated squares are 1 squared and 6 squared, yielding F4 = 37. Final R1 is 24, F1 is 6, F2 is 36, and F3 is zero. Two load/store FUs provide two address units, while memory itself remains single-ported.

Test case #02: widths 1 and 2

Issue width 1 - 47 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-7	8	9
Mult.d F2 F1 F0	2	9-23	-	24	25
Add.d F3 F3 F2	3	25-27	-	28	29
Mult.d F2 F1 F1	4	9-23	-	25	30
Ld F1 0(R1)	5	6	8-12	13	31
Addi R1 R1 4	6	7	-	9	32
Add.d F4 F2 F4	7	26-28	-	29	33
Addi R1 R1 8	8	10	-	11	34
Ld F1 0(R1)	9	12	13-17	18	35
Mult.d F2 F1 F0	10	19-33	-	34	36
Add.d F3 F3 F2	11	35-37	-	38	39
Mult.d F2 F1 F1	12	19-33	-	35	40
Ld F1 0(R1)	13	14	18-22	23	41
Mult.d F2 F1 F0	19	24-38	-	39	42
Add.d F3 F3 F2	25	40-42	-	43	44
Mult.d F2 F1 F1	26	27-41	-	42	45
Add.d F4 F2 F4	27	43-45	-	46	47

Issue width 2 - 46 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-7	8	9
Mult.d F2 F1 F0	1	9-23	-	24	25
Add.d F3 F3 F2	2	25-27	-	28	29
Mult.d F2 F1 F1	2	9-23	-	25	30
Ld F1 0(R1)	3	4	8-12	13	31
Addi R1 R1 4	3	4	-	5	32
Add.d F4 F2 F4	4	26-28	-	29	33
Addi R1 R1 8	4	6	-	7	34
Ld F1 0(R1)	5	8	13-17	18	35
Mult.d F2 F1 F0	9	19-33	-	34	36
Add.d F3 F3 F2	9	35-37	-	38	39
Mult.d F2 F1 F1	10	19-33	-	35	40
Ld F1 0(R1)	10	11	18-22	23	41
Mult.d F2 F1 F0	19	24-38	-	39	42
Add.d F3 F3 F2	25	40-42	-	43	44
Mult.d F2 F1 F1	25	26-40	-	41	45
Add.d F4 F2 F4	26	42-44	-	45	46

I measured these timings using sample_02.in at issue widths 1 and 2. EX/MEM are inclusive cycle ranges. All 17 instructions commit in program order.

Test case #02: widths 3 and 4

Issue width 3 - 46 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-7	8	9
Mult.d F2 F1 F0	1	9-23	-	24	25
Add.d F3 F3 F2	1	25-27	-	28	29
Mult.d F2 F1 F1	2	9-23	-	25	30
Ld F1 0(R1)	2	3	8-12	13	31
Addi R1 R1 4	2	3	-	4	32
Add.d F4 F2 F4	3	26-28	-	29	33
Addi R1 R1 8	3	5	-	6	34
Ld F1 0(R1)	3	7	13-17	18	35
Mult.d F2 F1 F0	9	19-33	-	34	36
Add.d F3 F3 F2	9	35-37	-	38	39
Mult.d F2 F1 F1	9	19-33	-	35	40
Ld F1 0(R1)	10	11	18-22	23	41
Mult.d F2 F1 F0	19	24-38	-	39	42
Add.d F3 F3 F2	25	40-42	-	43	44
Mult.d F2 F1 F1	25	26-40	-	41	45
Add.d F4 F2 F4	26	42-44	-	45	46

Issue width 4 - 46 cycles

Instruction	ISSUE	EX	MEM	WB	COMMIT
Ld F1 0(R1)	1	2	3-7	8	9
Mult.d F2 F1 F0	1	9-23	-	24	25
Add.d F3 F3 F2	1	25-27	-	28	29
Mult.d F2 F1 F1	1	9-23	-	25	30
Ld F1 0(R1)	2	3	8-12	13	31
Addi R1 R1 4	2	3	-	4	32
Add.d F4 F2 F4	2	26-28	-	29	33
Addi R1 R1 8	2	5	-	6	34
Ld F1 0(R1)	3	7	13-17	18	35
Mult.d F2 F1 F0	9	19-33	-	34	36
Add.d F3 F3 F2	9	35-37	-	38	39
Mult.d F2 F1 F1	9	19-33	-	35	40
Ld F1 0(R1)	9	10	18-22	23	41
Mult.d F2 F1 F0	19	24-38	-	39	42
Add.d F3 F3 F2	25	40-42	-	43	44
Mult.d F2 F1 F1	25	26-40	-	41	45
Add.d F4 F2 F4	26	42-44	-	45	46

I measured these timings using sample_02.in at issue widths 3 and 4. EX/MEM are inclusive cycle ranges. All 17 instructions commit in program order.

4 Correctness and recovery evidence

The regression suite contains **38 passing unittest methods**. These include 64 benchmark/width combinations, 120 general randomized sequential-oracle comparisons (seed 20261025), and 40 additional overlapping-memory comparisons (seed 180032). The new tests also cover address 18, binary32 forwarding, byte-range replay and strict JSON output. Nested width checks are not separate unittest methods.

Check	Evidence
Same-cycle RAW	Dependent instructions issue together; consumer EX waits until after producer WB.
WAW/WAR and R0	Hand-calculated rename benchmark and randomized register reuse; R0 always zero.
Structural hazards	Full RS/ROB stops later issue slots; no younger instruction bypasses the stall.
Resource timing	FP starts each cycle; integer EX is serialized; one CDB, one commit, one memory port.
Branch prediction	Taken/not-taken paths, BTB collisions, nested redirects and exact n+2 refetch.
Memory ordering	Youngest source per byte, partial overlaps, late store data and unknown-address replay.
Memory compatibility	Address 18 and binary32 transfers, including rounded forwarding and exact raw-byte checks.
Precise state	Wrong-path stores and completed results are squashed; invalid wrong-path accesses are harmless.
Parser and compatibility	Invalid widths/addresses rejected; legacy starter byte offsets match the normalized input.

Selected width-4 trace evidence

The memory-replay benchmark records 1 replay and finishes in 25 cycles. The speculative load is present in the trace as squashed; a later dynamic instance commits the correct value.

ID	Instruction	EX	MEM	WB	COMMIT	Status
1	Addi R1 R0 4	2-13	-	14	15	committed
2	Sd F1 0(R1)	15	16	-	16	committed
3	Ld F2 4(R0)	2	3	4	-	squashed
4	Add.d F3 F2 F2	5-7	-	8	-	squashed
5	Ld F2 4(R0)	18	19	20	21	committed
6	Add.d F3 F2 F2	21-23	-	24	25	committed

The forwarding benchmark produces F3=7 and records one forwarded load; its MEM start equals its MEM end. The branch-squash benchmark finishes with R1=1, R2=7, R3=8 and Mem[0]=3; the speculative store of F1=9 and register writes of 99 are removed. I checked the dynamic instruction IDs and squash reasons in the simulator output to verify recovery.

5 Reproduction and limitations

Run the simulator and sample workloads

```
cd tomasulo
python3 code/main.py
python3 code/main.py inputs/sample_01.in --issue-width 4
python3 code/main.py inputs/sample_02.in --issue-width 2 --json result.json
```

Run these commands from the repository root. Python 3.10 or later is required; the simulator uses only the standard library. The default example in **code/code.in** sets issue width to 2. An input without an Issue width line defaults to 1. I included both sample inputs so their timing tables can be reproduced at widths 1 through 4.

Compatibility decisions

I use word branch offsets: byte target = byte PC + 4 + 4 x offset. For older inputs that express offsets in bytes, I added **--legacy-branch-bytes**. For example, a byte offset of -12 corresponds to a word offset of -3. I also accept the assignment's unaligned load at address 18.

Model limits and what is not implemented

No unresolved functional failure was observed in the supplied tests and documented extensions. Memory initialization, loads, stores and forwarding now use binary32 values; FP register initialization and .d arithmetic retain Python precision. Rounding-mode controls, FP status flags and fixed-width integer overflow are not modeled. There are no caches, OS services, interrupts or exception handlers. Invalid committing memory accesses raise an error; the configurable cycle limit stops infinite loops.

The model fixes CDB width and commit width at one. Forwarding shares the single memory path. Reservation stations are freed at EX dispatch. Stores begin memory access only at the ROB head and commit at the end of that access. These choices are explicit so that timing comparisons can be reproduced. Branch prediction state is updated at EX resolution; speculative predictor updates are not rolled back, while architectural state is recovered.

Readable memory output contains nonzero binary32 views at aligned starts and explicitly initialized or committed-store starts. Those views may overlap. JSON includes the authoritative 256-byte image in **memory_bytes**; nonfinite numbers are encoded as the strings NaN, Infinity or -Infinity. Little-endian storage and no extra unaligned-access latency are documented conventions.

Repository contents

I organized the assignment under **tomasulo/**. I included the simulator in **code/**, the two report examples in **inputs/**, the verification suite in **tests/**, and saved traces and summaries in **results/**. I also included this PDF, its source in **tools/**, source snapshots in **report_assets/**, supplied materials in **original/**, earlier comparison files in **revision_before/**, the source diff, and attribution notes.

References

[1] Supplied programming-tomasulo.pdf, four PDF pages: processor, ISA, input/output and report requirements.
[2] Supplied Sample_Tomasulo_Assignment_Report.pdf, 21 PDF pages: report structure and Test Cases 01/02.
[3] Supplied tomasulo-python.zip, original Python simulator, source headers attributed to Qing, May 2017.